

Smart MCQ Solver Challenge

Project Report

BSDA2001P : Introduction to DL and Gen AI

Submitted by

Sharan Kumar E

Student ID: 24F3004935

Indian Institute of Technology Madras

May 2026

1. Abstract / Executive Summary

This project develops a deep learning system for the Smart MCQ Solver Challenge to answer science multiple-choice questions. A transformer model was first built from scratch to understand transformer architectures, followed by fine-tuning pretrained RoBERTa-large and DeBERTa models. To improve factual reasoning, a Retrieval-Augmented Generation (RAG) pipeline retrieves relevant passages from a STEM Wikipedia corpus and combines them with the question before prediction. Performance was evaluated using Accuracy, Macro F1-score, and MAP@3, demonstrating that integrating transformer models with external knowledge retrieval improves science MCQ answering.

2. Introduction

2.1 Problem Statement

The Smart MCQ Solver Challenge focuses on answering science multiple-choice questions by predicting the top three most probable answers (MAP@3). This project explores a custom transformer, a pretrained transformer, and a Lexical RAG pipeline to improve prediction accuracy using relevant scientific context.

2.2 Project objective

The primary objectives of this project are:

- Develop a transformer model from scratch for science MCQ classification.
- Fine-tune the Facebook RoBERTa-large model to improve prediction performance.
- Integrate a lexical RAG pipeline using the STEM Wikipedia corpus.
- Evaluate the models using Accuracy, Precision, Macro F1-score, and MAP@3.
- Analyze the results and identify potential future improvements.

2.3 Report Organization

This report is organized into eight sections. Section 3 describes the datasets and preprocessing steps. Section 4 explains the tokenization strategy. Section 5 presents the models and training process. Section 6 discusses the evaluation metrics and results. Section 7 provides error analysis and limitations, while Section 8 concludes the project and suggests future improvements.

3. Dataset & Preprocessing

3.1 Dataset Description

This project uses the provided training and test datasets, along with an external STEM Wikipedia corpus for the RAG pipeline to retrieve relevant scientific context.

Dataset	Purpose	Description
train.csv	Model Training	Contains science multiple-choice questions, five answer choices (A–E), and the corresponding correct answer label.
test.csv	Model Testing	Contains science multiple-choice questions without answer labels. The trained model predicts the top three most probable answers for submission.
STEM Wikipedia Corpus	Knowledge Retrieval	Contains scientific articles used by the lexical RAG pipeline to retrieve relevant context for answering science multiple-choice questions.

3.2 Exploratory Data Analysis (EDA)

Before training, exploratory data analysis (EDA) was performed to examine the dataset structure, class distribution, prompt length, missing values, and duplicate records. This helped verify data quality before preprocessing and model training.

3.2.1 Dataset Overview

Property	Value
Training Samples	2,000
Test Samples	500
Features in train.csv	8
Features in test.csv	7

The training dataset contains an ID, a question, five answer choices (A–E), and the correct answer, while the test dataset has the same structure without the answer column. No missing values were found in either dataset.

Dataset	Total	Duplicate Prompts	Unique Prompts
Training	2000	242	1758
Test	500	8	492

Duplicate prompt analysis identified 242 repeated prompts in the training dataset and 8 in the test dataset. These records were retained because they represent valid questions with different answer options and contribute useful training information.

3.3 Data Preprocessing

Before training, answer labels (A–E) were encoded as numerical values (0–4), duplicate questions were removed, and each question was combined with its five answer choices into a single input. The data was then split into training and validation sets using an 80:20 stratified split.

4. Tokenization Strategy

The final model uses the pretrained DeBERTa SentencePiece tokenizer to convert input text into subword tokens, enabling efficient handling of scientific and uncommon words while remaining compatible with the pretrained model.

4.1. Justification

SentencePiece-based tokenization effectively handles rare words by splitting them into meaningful subword units instead of treating them as unknown tokens. This is particularly useful for science multiple-choice questions, which often contain specialized technical terms.

4.2 Implementation Details

Tokenization was implemented using the **Hugging Face Transformers library**. Input sequences were converted into token IDs and attention masks, with padding and truncation applied for efficient batch processing.

4.3 Experimental Comparison

A custom word-level tokenizer was used for the scratch transformer, while RoBERTa and DeBERTa used their pretrained Hugging Face tokenizers. The DeBERTa tokenizer was selected for the final system due to its superior performance.

5. Models

5.1 Model 1 - Fine Tuned Transformer Model

5.1.1 Architecture

The first pretrained model used in this project was **FacebookAI/RoBERTa-large**. It consists of **24 transformer encoder layers** with a hidden size of **1024** and a classification head configured for **five output classes**.

5.1.2 Salient Points

RoBERTa-large uses a transformer encoder with a self-attention mechanism to capture contextual relationships between words. This enables effective understanding of science multiple-choice questions.

5.1.3 Fine-Tuning Strategy

The pretrained RoBERTa-large model was fine-tuned on the training dataset by replacing the final classification layer with a five-class output layer. All model parameters were updated during training.

5.2 Model 2 - Custom DL Model

5.2.1 Architecture

A custom transformer classifier was built in PyTorch with an embedding layer, sinusoidal positional encoding, two transformer encoder layers, masked mean pooling, dropout, and a final classification layer to predict the five answer classes.

5.2.2 Salient Points

- Built entirely from scratch without pretrained weights.
- Uses sinusoidal positional encoding to retain sequence information.
- Employs multi-head self-attention to capture contextual relationships
- Add & Layer Normalization improves training stability and helps preserve information through residual connections. .
- Uses masked mean pooling to obtain a fixed-length sentence representation.
- Predicts one of the five answer classes through a linear classifier.

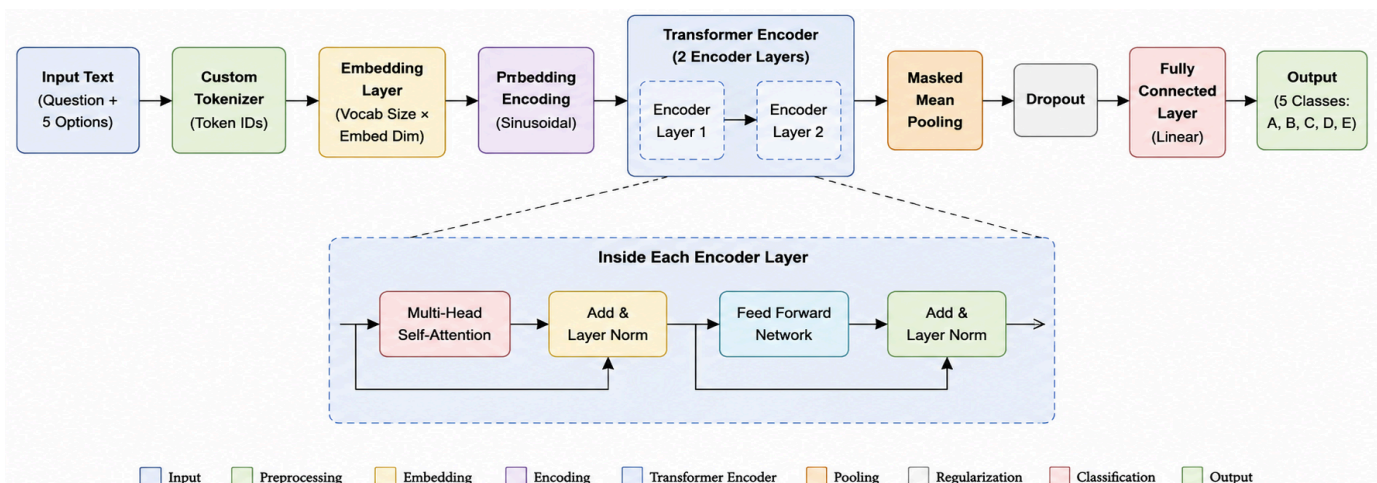


Figure 5.2: Architecture of the Custom Transformer Classifier

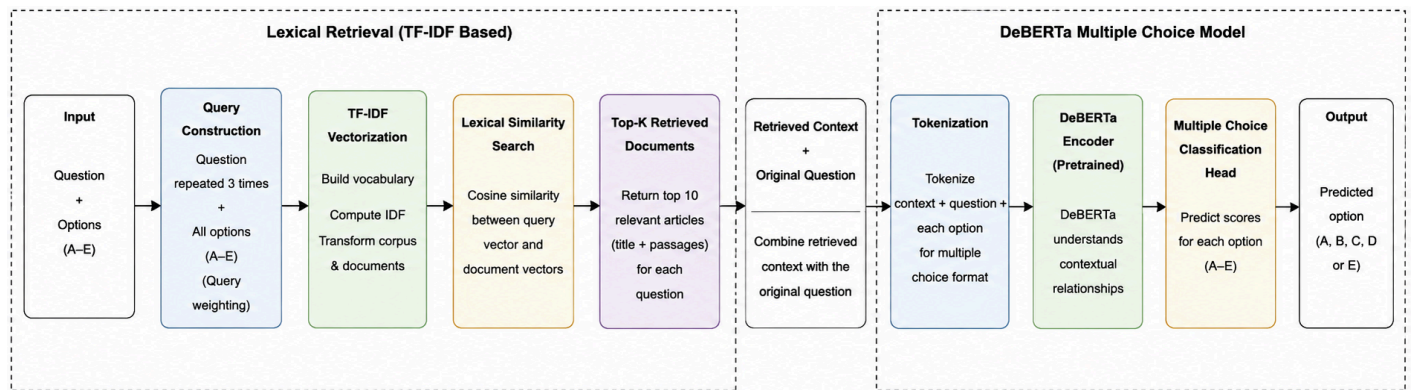
5.3 Model 3 - Lexical RAG (TF-IDF)

5.3.1 Architecture

The final model combines TF-IDF-based lexical retrieval with a pretrained DeBERTa model. Retrieved context is merged with the question and passed to DeBERTa to predict the most probable answer.

5.3.2 Salient Points

- Retrieves relevant context using TF-IDF and cosine similarity.
- Combines retrieved context with the input question.
- Uses a pretrained DeBERTa model for contextual understanding.
- Predicts the correct answer through a multiple-choice classification head.



6. Performance & Comparative Analysis

6.1 Evaluation Metrics

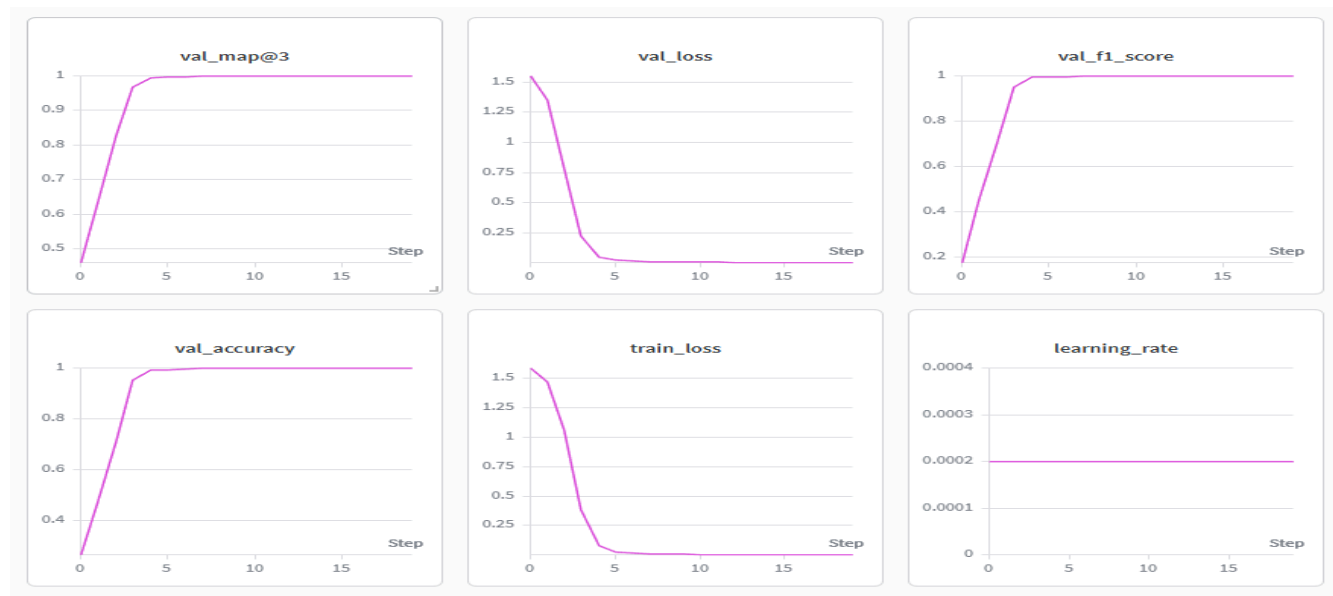
The following metrics were used to evaluate the performance of the models:

- **Accuracy:** Measures the percentage of correctly classified questions.
- **F1-Score:** Evaluates the balance between precision and recall, making it suitable for multi-class classification.
- **Precision:** Measures the proportion of correct positive predictions among all predicted positives.
- **Loss:** Represents the model's prediction error during training and validation. Lower loss indicates better learning performance.

6.2 Training Performance

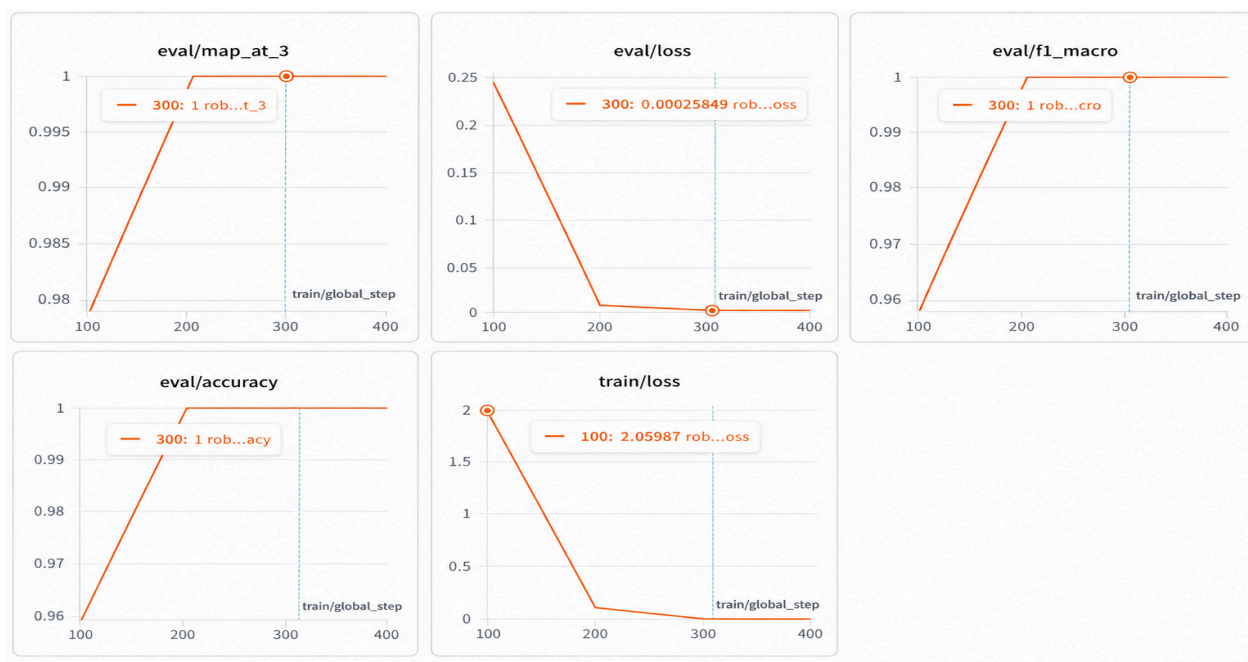
Scratch model

The scratch transformer model achieved stable convergence during training. Training and validation loss decreased consistently, while validation accuracy, F1-score, and MAP@3 improved across epochs, demonstrating effective learning.



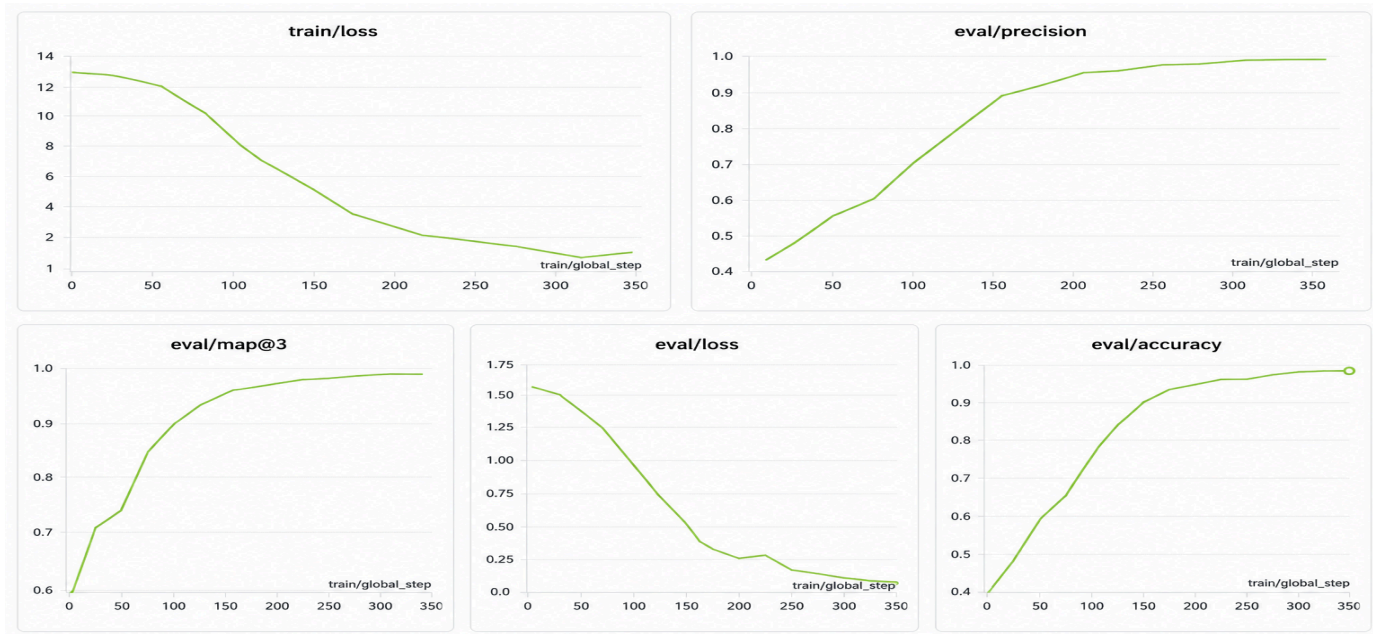
Pre-trained Model

The RoBERTa-large model showed stable convergence with decreasing training and validation loss. Validation accuracy and F1-score improved consistently throughout training.



Lexical RAG (DeBerta-large + tf-idf)

The DeBERTa with Lexical RAG model achieved the best validation performance. The training curves indicate stable convergence with consistent improvements in evaluation metrics.



6.3 Comparative report

Model	Accuracy	Loss	Training Time	Computational cost	Remarks
Custom Transformer	0.9886	0.0008	Moderate	Moderate	Achieved competitive performance despite being trained from scratch.
RoBERTa-large	1.0000	0.00121	High	High	Best overall performer by leveraging retrieved contextual information to improve answer prediction.
DeBERTa + Lexical RAG	0.9801	2.44597	High for deberta-large	High for deberta-large	Best overall performer by leveraging retrieved contextual information to improve answer prediction.

6.4 Kaggle Performance

Model	Kaggle Score (map@3)
Custom Transformer	0.73690
RoBERTa-large	0.74729
DeBERTa + Lexical RAG	0.75644

Among all the experimented models, **DeBERTa + Lexical RAG** achieved the best Kaggle score (**0.75644**) and was therefore selected as the final model.

7. Conclusion & Future Work

7.1 Key Learnings

Gained practical experience in fine-tuning transformer models, building a custom transformer, applying lexical RAG, and evaluating deep learning models.

7.2 Challenges Faced

The main challenges included GPU memory limitations during training, identifying optimal hyperparameters, handling long context sequences within the model's maximum input length, improving retrieval quality, and effectively integrating retrieved knowledge with the pretrained DeBERTa model.

7.3 Areas for Improvement

Future work includes replacing the TF-IDF lexical retriever with semantic retrieval, developing a Hybrid RAG framework by combining lexical and semantic retrieval, experimenting with stronger embedding models (BGE, BAAI, SentenceTransformers), and further hyperparameter tuning and ensemble methods to improve retrieval quality and overall prediction accuracy.

8. References

1. Hugging Face Transformers Documentation.
2. PyTorch Documentation.
3. Scikit-learn Documentation (TF-IDF and Cosine Similarity).
4. Weights & Biases Documentation.
5. Kaggle Competition Dataset and Evaluation Platform.
6. External Dataset: *All Paraphs Parsed Expanded*. Kaggle. [All Paraphs Parsed Expanded](#)
7. External Dataset: *STEM Wiki Cohere No Embed*. Kaggle. [STEM Wiki Cohere No Emb](#)